DecisionTwin Architecture Research — Two-Page Summary

Project: DecisionTwin, an AI-assisted financial decision tool for HackPSU Fall 2026 (FNB Challenge, Nittany AI Challenge). **Date:** October 24, 2026. **Scope:** architecture and documentation only; no application code was written.

What was done

The brief called for six stage agents and a supervisor. The chat environment can't spawn subagents, so this ran as six scoped research passes plus one synthesis pass (disclosed in the README).

Verification first. From the official Devpost page: the entry must be *created* by Saturday midnight, the hard deadline is Sunday noon, and a five-minute video and public repo are required; judging covers Implementation, Clarity, Creativity, Growth and Impact. Gemini's structured-output and PDF docs were checked, and the loan formulas were recomputed in Python and match CFPB's published examples to the cent ($20,000 at 4.75% for 60 months: $375.14/month, $2,508.29 interest).

Shared contracts first. The supervisor defined the shared data objects and REST API before research began; cross-agent reviews later revised them (e.g., adding gross income for debt-to-income).

Six stage investigations. Each compared 3–5 alternatives in a weighted matrix with a sensitivity test, then chose one:

1. **Adaptive questioning:** a small typed dependency graph plus an append-only fact ledger. Questions are ranked by how much the answer would change the results, minus effort and privacy cost, and stop once the minimum facts are in. No graph database.
2. **Document intelligence:** PyMuPDF reads the file, Gemini extracts each term with a verbatim quote and page, and a verifier checks the quote, the TILA arithmetic and the APR. The user confirms every value; manual entry is the fallback.
3. **Financial engine:** exact decimal arithmetic for payment, total cost, APR, LTV, DTI, monthly cash left over and savings runway. Flags carry CFPB citations; nothing is labeled "safe" or "unsafe."
4. **Risk simulation:** named deterministic shocks (e.g., losing three weeks of income twice a year, the CFPB median) and a "largest income drop you could absorb" measure. No Monte Carlo or distress scores in the MVP: no data supports them.
5. **Alternatives:** combinations of down payment, term, add-ons and competing offers run through the same engine; only non-dominated (Pareto) options are shown, as trade-offs rather than a pick.
6. **Explainable experience:** every number shows its source (you, your document, a calculation, or an assumption). Explanations are templates; optional AI wording is kept only if every number in it matches the results.

Final decision. The supervisor cut what didn't earn its complexity (graph DB, vector store, agent framework, microservices, mandatory cloud). Result: Next.js frontend, FastAPI backend with six in-process modules, one SQLite database, and three bounded LLM calls, each with a non-AI fallback. Three decisions remain open with owners: Gemini model/quota, the depreciation assumption, and a hypothetical lower-rate alternative.

Team plan. Dev 1: engine, scenarios, alternatives. Dev 2: intake and documents. Dev 3: frontend. Dev 4: API, explanations, demo, submission. Build the form → engine → dashboard slice first; code freeze Sunday 09:00; submit by 11:00.

What each file contains

Stage reports are `stage_0X_architecture.md` (23-section research report) and `stage_0X_technical_stack.docx`; supervisor files are in `supervisor/`.

File	Contents
`README.md`	Reading order, process disclosure, verification status, full deliverable index
`SUMMARY.md`	This two-page summary
`stage-01-knowledge-graph/` (.md)	Financial ontology, question-ranking formula, stopping rules, contradiction rules, three persona question paths
`stage-02-document-intelligence/` (.md)	Upload-to-verified-offer pipeline, extraction schema, quote and arithmetic checks, prompt-injection defences
`stage-03-financial-modeling/` (.md)	Formulas F01–F12, rounding and precision policy, missing-value handling, CFPB reference tests
`stage-04-risk-simulation/` (.md)	Scenario catalog, monthly cash-flow simulation, sensitivity and break-even methods, why no Monte Carlo in the MVP
`stage-05-counterfactual-`	Levers, alternative grid, Pareto filtering, lever ranking, counterfactual-vs-causal distinction

File	Contents
`intelligence/` (.md)	
`stage-06-explainable-ai/` (.md)	Explanation architecture and validation gate, user journey, screens, visualisations, accessibility
Each stage folder (.docx)	Word spec per stage: technologies and why, costs, milestones, testing
`system_architecture.md`	Final architecture, module ownership, AI boundaries, security, deployment, review outcomes
`master_technical_specification.docx`	Consolidated Word spec for all four developers
`api_and_data_contracts.md`	Shared JSON schemas, REST endpoints, error codes, validation and compatibility rules
`team_execution_plan.md`	Deadlines, roles, hour-by-hour plan, checkpoints, cut list, demo script
`architecture_decisions.md`	Fifteen cross-cutting decision records with options, evidence and risks
`integration_risk_register.md`	Cross-stage dependencies, open questions, 16 rated risks with owners and tests
`mvp_acceptance_plan.md`	Acceptance tests A-01 to A-19, evaluation metrics, and what is explicitly not claimed
`diagrams/` (9 .mmd)	Editable Mermaid diagrams: system, data flow, ontology, question workflow, document flow, simulation, sequence, deployment, error recovery

All Word files were render-checked, and all nine diagrams pass the Mermaid parser.